

Persistent Contracts for Managing Long-Horizon Language-Agent Work

Anonymous Authors

August 30, 2026

Abstract

Meta-agents increasingly decompose objectives, dispatch workers, revise plans, and supervise compound language-agent systems. The difficult failure mode is often not a single bad action but gradual loss of intent, authority, evidence, or dependency validity across many runs and fresh contexts. We present AI-Native Workflow, a package-resident management protocol that separates human governance from machine execution. A human approves a multi-phase Plan; a persistent Main coordinates Theory and Experiment workstreams; each Phase is governed by one durable multi-view contract (DIC) whose Requests state concrete goals and whose Plan records prompt dependencies, parallel subagents, merge points, and repair branches. Disposable execution prompt stacks (EPS) instantiate those planned nodes just in time. Evidence gates return accept, repair, replan, or escalate. Under complete dependency declarations and version-bound evidence, we show that a repair need invalidate only the dynamic dependency cone and that this cone is worst-case tight; imperfect verification admits a simple survival bound, while evidence aliasing yields an impossibility result. The reference implementation records reconstructable lineage and periodic human checkpoints. We pre-register a resource-constrained proof-of-concept evaluation on five Harbor tasks using ChatGPT as Main and Claude Code as bounded executor. No comparative benchmark result is claimed before those runs are completed; the present paper contributes the system, formal analysis, and a falsifiable evaluation design.

1 Introduction

Long-running language-agent work increasingly resembles management rather than a single call to a model. A manager interprets an objective, decomposes it, delegates bounded work, integrates outputs, checks evidence, and decides whether a failure should be repaired locally or should change the plan. Existing work already demonstrates planning and search [Yao et al., 2023b,a, Besta et al., 2024, Prasad et al., 2024, Zhou et al., 2024], role-based compound agents [Wu et al., 2023, Li et al., 2023, Chen et al., 2024, Hong et al., 2024], externalized context [Packer et al., 2023, Wang et al., 2024], and automated harness optimization [Khatab et al., 2024, Hu et al., 2025]. Accordingly, our claim is not that “agents can manage agents” or that hierarchy itself is new.

We focus on a narrower systems problem: *what durable management objects should survive when a task spans many model contexts, workers, files, tests, scientific claims, and human review cycles?* A local worker can be competent while the overall programme fails because the original request was silently weakened, a downstream artifact still depends on an invalidated upstream result, a verification signal is stale, or a manager changes a publication/release objective without human authority.

AI-Native Workflow treats these as control and state-management failures. Its canonical refinement chain is

$$\text{Goal} \succeq \text{Plan} \succeq \text{Phase} \succeq \text{DIC} \succeq \text{EPS} \succeq \text{Prompt/Run} \succeq \text{Action}.$$

A Plan is the human-approved programme. A Phase is a coherent machine milestone. A *durable intent contract* (DIC) is one multi-view Phase package: `README.md` gives the summary, `REQUESTS.md` enumerates concrete goals and acceptance targets, and `PLAN.md` records the actual prompt graph, dependencies, parallel subagent batches, merge nodes, and conditional branches. Optional architecture, test-matrix, and user-defined views add other angles on the same contract. An EPS does not replace this plan; it is a disposable instantiation of planned prompt/run nodes against the current repository state and attempt.

The design deliberately separates two timescales. Humans approve a Plan and review periodic or material checkpoints. Ordinary machine phases, prompt runs, local tests, and repairs do not each require approval. Main

retains integration and publication authority; Theory and Experiment are persistent specialist workstreams; coding or research subagents are bounded executors rather than independent authorities.

Our contributions are:

1. a concrete management protocol and reference runtime for persistent intent, bounded delegation, evidence lineage, periodic checkpoints, and explicit repair/replan/escalate outcomes;
2. a formal characterization of when refinement and dependency-local recovery are sound, including a worst-case tightness statement, an imperfect-verifier bound, and an evidence-observability limit;
3. a small, falsifiable workshop evaluation that compares a competent flat ChatGPT+Claude-Code workflow with the full hierarchy on five evolving/stateful/scientific Harbor tasks, rather than claiming performance from engineering unit tests.

2 Management problem and design principles

Persistent intent. A manager needs a stable description of what is being optimized, what may not change, and what evidence closes the task. Chat history is insufficient because fresh contexts and tool agents do not share a single reliable memory surface.

Bounded delegation. A worker should receive the smallest sufficient context, write scope, acceptance test, and return path. Delegation transfers execution, not the authority to redefine the objective.

Evidence before completion. “Done” is not a linguistic state. A request is closed only when its declared evidence/acceptance relation is satisfied. The framework therefore treats test outputs, artifact hashes, benchmark rewards, citations, and human decisions as first-class state.

Local repair before global restart. If a failure affects only a subset of declared dependencies, the manager should preserve independent work. Conversely, incomplete dependency declarations make locality unsafe.

Human governance at the right scale. Requiring approval after every phase defeats autonomous execution; never escalating lets the system silently change objectives or external state. The default is Plan approval, periodic review after several phases, and immediate escalation for material objective, authority, protocol, scientific-claim, destructive/external, credential, paid-compute, release, or submission changes.

3 Contracted hierarchy

3.1 Plan, Phase, and multi-view DIC

A Plan is a DAG of machine-managed Phases plus human governance metadata. A Phase is not normally a chat session: one Phase can legitimately contain tens of prompt/runs and several subagents.

Each Phase has exactly one DIC package. The core views are complementary rather than sequential:

- `README.md`: general summary, current state, owner, authority, and read order;
- `REQUESTS.md`: specific goals and acceptance targets, independent of execution ordering;
- `PLAN.md`: durable execution graph—prompt IDs, dependencies, parallel batches, assigned subagents, merge/review nodes, and conditional repair branches.

`ARCHITECTURE.md` specifies a target repository/system architecture only when the Phase changes it; `TEST_MATRIX.md` specifies concrete software, theory-audit, or experiment combinations. Users may add arbitrary registered DIC views such as a theorem map, data contract, venue requirements, or release contract.

This distinction matters operationally. If a proof route fails but the Phase still seeks the same theorem-or-obstruction answer, the durable Request remains unchanged while the execution plan/EPs can be repaired. If the theorem target itself must be weakened, the DIC changes and the alteration is visible.

3.2 EPS and prompt/run materialization

The EPS is a just-in-time realization of the durable plan. For a node p , the EPS records the current attempt, exact context inputs, executor/model profile, dependencies, output path, local acceptance, and stop conditions. A failed attempt can be superseded by a new EPS instance without overwriting the original trace. Stable prompt IDs remain useful for dependencies and postmortem analysis even though the human does not approve each prompt.

Parallelism is explicit. Two workers may run concurrently only when their dependencies are satisfied, write scopes are compatible, their failures can be recovered independently, and a merge node is named. The manager remains responsible for integration.

3.3 Persistent state and reports

`.ai-workflow/STATE.json` is a small canonical index for active Plan/Phase state, roles, checkpoints, escalations, DIC/EPS paths, and evidence pointers. Substantive outputs remain in their natural repository locations. Every Phase has a mandatory flat `reports/` surface, capped at ten files by default, combining concise human synthesis with key merged raw results. “Merged raw” permits lossless collation and provenance but not averaging, favorable-case selection, or paraphrase of failed outputs.

This design allows a fresh Main to reconstruct authority and progress from the package alone rather than relying on an earlier conversation.

4 Verification-gated management

Let a verifier consume the relevant DIC, current state, and evidence and return

$$\mathcal{V} \in \{\text{accept, repair, replan, escalate}\}.$$

Accept closes the local request/phase boundary. **Repair** preserves the durable contract and regenerates/re-executes affected operations. **Replan** changes the durable execution decomposition while staying within authorized intent. **Escalate** stops autonomy at a human boundary.

Verification is layered rather than synonymous with a separate “Verifier agent.” Local checks occur near the action (unit tests, compilation, schema validation, artifact reopen, numerical sanity checks). Milestone checks summarize several prompt nodes or close a DIC Request. Fresh-context independent review is reserved for theorem-level claims, frozen empirical evidence, or release/submission boundaries where epistemic independence is worth the cost. Reflection-style additional inference [Shinn et al., 2023] is therefore only one possible implementation of a verification gate.

5 Formal analysis

5.1 Refinement and composition

Associate every hierarchy object X with a set of admitted traces $\llbracket X \rrbracket$. Given an abstraction map ρ , define $X \preceq Y$ when

$$\rho(\llbracket X \rrbracket) \subseteq \llbracket Y \rrbracket.$$

A chain of hierarchy labels alone does not preserve intent: the abstraction maps and inclusions must actually be sound. For a Plan DAG, standard contract composition follows if each Phase satisfies a sound pre/post contract, predecessor postconditions imply successor preconditions, unordered writes are noninterfering, and terminal postconditions imply the global goal. We include this result to expose assumptions, not as a novel program-logic theorem [Hoare, 1969].

5.2 Dependency-local recovery

Let Phase or prompt node v declare read keys R_v and write keys W_v ; dependency edges include every possible information flow. If repair changes keys Δ , define

$$B(\Delta) = \{v : R_v \cap \Delta \neq \emptyset\}, \quad \mathcal{I}(\Delta) = \text{Reach}(B(\Delta)).$$

Theorem 1 (Dependency-local recovery). *If outputs/evidence depend only on declared inputs, predecessor outputs, recorded tool/version parameters and recorded randomness; dependencies are complete; and evidence is bound to artifact versions, then completed outputs outside $\mathcal{I}(\Delta)$ remain contract-valid after the repair.*

Proof sketch. A node outside the cone neither reads a changed key nor depends on a changed predecessor. Topological induction therefore preserves all declared inputs and the validity of its version-bound evidence. The detailed statement/proof is in the technical report. $\square$

The cone is worst-case tight given footprint information alone: a direct reader can copy a changed bit and every descendant can propagate it. This is an adaptation of dependency/incremental-build locality [Mokhov et al., 2018, Acar, 2009], not a claim that dependency graphs are new.

5.3 Imperfect verification and observability

If a persisting observable fault is missed at each gate with conditional probability at most $\alpha < 1$, survival through k gates is at most α^k and expected detection delay is at most $1/(1 - \alpha)$ gate intervals. A separate stationary checkpoint-cost model yields a square-root trade-off between checking cost and expected repair cost, analogous to checkpoint/restart analysis [Young, 1974, Daly, 2006].

There is also a hard limit. Let $O(s)$ be the evidence observation and $G(s)$ the goal predicate. If $O(s_+) = O(s_-)$ but $G(s_+) \neq G(s_-)$, every verifier using only O must make the same decision on both states and therefore incur either a false acceptance or false rejection. More management hierarchy cannot recover information missing from the evidence channel.

6 Reference implementation

The standard-library runtime records Plan approval, dependency-aware phase progression, one multi-view DIC per Phase, EPS prompt/run lineage, role normalization, periodic checkpoints, material escalations, verification outcomes, and package-only recovery. Optional `minor/`, `research/`, `verification/`, and `checkpoints/` lanes use `off|auto|on; reports/` is mandatory and flat. Legacy Orchestrator/Researcher terminology is retained only as compatibility aliases to Main/Theory capabilities.

The current conformance suite tests one Plan authorizing several phases, autonomous ordinary progression, checkpoint triggering, material escalation, reconstructable lineage, Main/Theory/Experiment ownership, package-only recovery, multi-view DIC/custom views, parallel EPS nodes, report invariants, and v5.6.x migration surfaces. These are engineering tests: they establish that the protocol is implemented, not that it improves agent performance.

7 Targeted workshop evaluation

7.1 Resource-constrained design

We pre-register a deliberately small paired study because the available evaluation budget is one ChatGPT Plus account and two Claude Pro accounts. ChatGPT is Main in every run. Claude Code authenticated through Pro A is the primary bounded executor; Pro B is reserved for pre-declared replication on the two most mechanism-relevant tasks. We compare only two conditions so that the study measures a meaningful system difference rather than spreading credit across a large ablation matrix.

F (Flat) is a competent baseline using the same ChatGPT Main + Claude Code stack. Main may make one ordinary plan, inspect tests, debug, and iterate, but does not instantiate DIC/EPS lineage, a persistent Request

Table 1: Frozen candidate task set. No score in this table is a result.

ID	Task	Intended stressor
T1	env_manager	5-step easier evolving-spec control; hierarchy overhead
T2	database_migration	5-step dependencies, rollback, backfills, cycle handling
T3	recli	8-step hard evolving CLI; late regression/architecture erosion
T4	live-database-cutover	live state, consistency, monitoring, safe cutover
T5	certified-sparse-regression	scientific optimization/certification transfer

ledger, dependency-local invalidation, or the four-outcome verification state machine. **W (Workflow)** uses the full package hierarchy. Thus F is not a one-shot straw man.

T1–T3 come from SlopCodeBench’s iterative specification-refinement tasks [Orlanski et al., 2026]; T4 is a current Terminal-Bench task [Merrill et al., 2026]; T5 is from Terminal-Bench-Science [Terminal-Bench Science Team, 2026]. Primary runs consist of F/W on all five tasks with Claude Pro A (10 runs). T2 and T3 receive F/W replications with Pro B (4 runs), for 14 scored runs total. Exact task versions are locked before execution and oracles are used only for isolated environment qualification.

Primary evidence is the official reward, SlopCodeBench strict pass rate by checkpoint, regression/erosion, re-work/repair scope, model-run counts, Main turns, and wall time. Because subscription UIs may not expose reliable token counts, we do not claim strict token matching; both conditions receive the same benchmark timeout and tool/account surface, and resource differences are reported. With five task families, the analysis is descriptive: all runs, paired differences, checkpoint curves, and independent-account replications are shown rather than converted into broad population-level p -value claims.

Falsifiability. The workflow can fail this test. T1 may expose pure overhead. T2/T3 may show no improvement in late-checkpoint retention. T4 may demonstrate that explicit verification does not prevent unsafe state transitions. T5 may show that the architecture does not transfer to scientific workflows. Null or negative outcomes are retained in the evidence store and manuscript.

Current evidence boundary. At the time of this draft, the 14 scored runs have not been executed. No table or abstract claim should be upgraded from “pre-registered evaluation” to “improves” until generated directly from raw Harbor/verifier evidence.

8 Related work

Our hierarchy is compatible with plan/search agents such as ReAct, Tree/Graph of Thoughts, ReWOO, ADaPT, and LATS [Yao et al., 2023b,a, Besta et al., 2024, Xu et al., 2023, Prasad et al., 2024, Zhou et al., 2024]. Compound-agent frameworks demonstrate role specialization and inter-agent communication [Wu et al., 2023, Li et al., 2023, Chen et al., 2024, Hong et al., 2024]; our focus is the persistent authority/evidence layer around such execution. MemGPT and HiAgent externalize or hierarchize context [Packer et al., 2023, Wang et al., 2024]; here external state is tied to acceptance and invalidation. DSPy and automated agent design optimize prompts/graphs [Khatab et al., 2024, Hu et al., 2025]; AI-Native Workflow governs how a chosen or evolving harness is authorized, audited, and repaired. Verification and oversight work motivates process checks, debate, recursive oversight, and weak-to-strong supervision [Lightman et al., 2023, Irving et al., 2018, Leike et al., 2018, Burns et al., 2024]; our contribution is to connect evidence gates to explicit recovery scope and governance outcomes.

9 Limitations

The protocol introduces overhead and can be counterproductive on short tasks. DIC/Plan quality remains model- and operator-dependent. Dependency-local recovery is unsound when hidden state or undeclared information flows exist. Verification rates may be correlated and evidence may be adversarial or simply irrelevant to the true goal. A package-resident state machine is not a sandbox or security boundary; agent-security benchmarks expose separate failure modes [Ruan et al., 2024, Debenedetti et al., 2024]. The current empirical programme is intentionally small

and cannot support a universal benchmark claim. Finally, using ChatGPT Main plus Claude Code makes the first study a system instantiation rather than a model-agnostic proof; cross-backend replication is future work.

10 Responsible-use statement

Meta-agent infrastructure can amplify both productive and harmful objectives by coordinating multiple agents, retaining long-lived state, and automating repair. The same management features that reduce accidental drift could make unwanted objectives more persistent. We therefore make external/destructive actions, credentials/private systems, paid compute, release/submission changes, and material scientific/objective changes explicit authority boundaries; preserve audit lineage and failed attempts; and recommend sandboxing and least privilege for executors. These controls reduce accidental misuse but do not guarantee alignment or security, and they should not be treated as authorization to deploy autonomous agents in high-stakes environments without domain-specific oversight.

11 Conclusion

Long-horizon agent management needs more than a stronger prompt to the next worker. AI-Native Workflow makes persistent intent, execution structure, evidence, recovery scope, and human authority explicit in the repository that the agents modify. The formal results identify the assumptions under which local recovery is safe and the evidence limitations that no hierarchy can remove. The next question is empirical and deliberately falsifiable: on a small set of evolving and stateful tasks, does this extra structure preserve more prior work and reduce regressions enough to justify its overhead?

References

- Umut A. Acar. *Self-Adjusting Computation*. Morgan & Claypool, 2009.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In *International Conference on Machine Learning*, 2024.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations*, 2024.
- John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, 2024.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.
- Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.

- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.
- Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: A research direction. *arXiv preprint arXiv:1811.07871*, 2018.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for mind exploration of large scale language model society. In *Advances in Neural Information Processing Systems*, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. In *Proceedings of the ACM on Programming Languages*, 2018.
- Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Aws Albarghouthi, and Frederic Sala. Slopcodibench: Measuring code erosion under iterative specification refinement. *arXiv preprint arXiv:2603.24755*, 2026. URL <https://arxiv.org/abs/2603.24755>.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, and Tushar Khot. Adapt: As-needed decomposition and planning with language models. In *Findings of the Association for Computational Linguistics: NAACL*, 2024.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Toolemu: Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Terminal-Bench Science Team. Terminal-bench science: Evaluating ai agents on computational workflows in the natural sciences. <https://github.com/harbor-framework/terminal-bench-science>, 2026.
- Zhenhailong Wang, Shaoguang Liu, Yixin Chen, Zhaoran Zheng, Mingyuan Wang, Kenji Kawaguchi, and Huaxiu Yao. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language models. *arXiv preprint*, 2024.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Binfeng Xu, Zhendong Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.
- John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*, 2024.